

Nome: Guilherme de Moraes China Costa

Data: 09/11/2025

Título: Engenharia Disciplinada de Softwares - TP1

Link do github com a implementação:

https://github.com/guilhermecosta/INFNET/tree/master/Engenharia_Disciplinada_de_Software/crud-produtos

Especificações do Sistema de Gerenciamento de Produtos

1. Visão Geral do Sistema

O Sistema de Gerenciamento de Produtos é uma aplicação CRUD (Create, Read, Update, Delete) que permite o gerenciamento completo de um catálogo de produtos. O sistema opera através de uma interface de linha de comando (CLI) e mantém os dados em memória durante a execução.

2. Especificações de Domínio

2.1 ProdutoId (Identificador de Produto)

Descrição: Identificador único e imutável para cada produto no sistema.

Comportamentos Esperados:

- DEVE gerar um identificador único utilizando UUID quando criado através do método gerar()
- DEVE aceitar um identificador existente através do método de(String)
- DEVE rejeitar valores nulos com IllegalArgumentException
- DEVE rejeitar strings vazias ou contendo apenas espaços com IllegalArgumentException
- DEVE manter imutabilidade após criação
- DEVE implementar igualdade baseada no valor do identificador
- DEVE produzir o mesmo hashCode para identificadores iguais

Invariantes:

- O valor do identificador nunca é nulo
- O valor do identificador nunca é vazio
- Uma vez criado, o identificador não pode ser modificado

2.2 NomeProduto (Nome do Produto)

Descrição: Representa o nome descritivo de um produto com validações de tamanho.

Comportamentos Esperados:

- DEVE aceitar nomes entre 3 e 100 caracteres (após remoção de espaços)
- DEVE remover espaços em branco no início e fim automaticamente
- DEVE rejeitar valores nulos com `IllegalArgumentException`
- DEVE rejeitar strings vazias ou apenas com espaços com `IllegalArgumentException`
- DEVE rejeitar nomes com menos de 3 caracteres com `IllegalArgumentException`
- DEVE rejeitar nomes com mais de 100 caracteres com `IllegalArgumentException`
- DEVE manter imutabilidade após criação
- DEVE implementar igualdade baseada no valor do nome

Invariantes:

- Tamanho mínimo: 3 caracteres
- Tamanho máximo: 100 caracteres
- Não contém espaços em branco no início ou fim
- Nunca é nulo ou vazio

Partições de Equivalência:

- **Válidas:**
 - Nomes com 3 caracteres (limite inferior)
 - Nomes entre 4 e 99 caracteres (faixa válida)
 - Nomes com 100 caracteres (limite superior)
- **Inválidas:**
 - null
 - String vazia
 - Apenas espaços em branco
 - 1-2 caracteres
 - Mais de 100 caracteres

Valores Limite:

- Limite inferior válido: "ABC" (3 caracteres)
- Limite inferior inválido: "AB" (2 caracteres)
- Limite superior válido: String com 100 caracteres
- Limite superior inválido: String com 101 caracteres

2.3 Preço (Preço do Produto)

Descrição: Representa o valor monetário de um produto com precisão de centavos.

Comportamentos Esperados:

- DEVE aceitar valores entre R\$ 0,00 e R\$ 999.999,99
- DEVE aceitar criação a partir de `BigDecimal` ou `double`
- DEVE arredondar valores para 2 casas decimais usando `HALF_UP`
- DEVE rejeitar valores nulos com `IllegalArgumentException`
- DEVE rejeitar valores negativos com `IllegalArgumentException`
- DEVE rejeitar valores acima de R\$ 999.999,99 com `IllegalArgumentException`
- DEVE manter exatamente 2 casas decimais após criação

- DEVE formatar para exibição como "R\$ X.XX"
- DEVE implementar igualdade baseada em comparação numérica

Invariantes:

- Valor nunca é nulo
- Valor sempre $\geq$ R\$ 0,00
- Valor sempre $\leq$ R\$ 999.999,99
- Sempre possui exatamente 2 casas decimais

Partições de Equivalência:

- **Válidas:**
 - R\$ 0,00 (limite inferior)
 - R\$ 0,01 a R\$ 999.999,98 (faixa válida)
 - R\$ 999.999,99 (limite superior)
- **Inválidas:**
 - null
 - Valores negativos
 - Valores acima de R\$ 999.999,99

Valores Limite:

- Limite inferior válido: R\$ 0,00
- Limite inferior inválido: R\$ -0,01
- Limite superior válido: R\$ 999.999,99
- Limite superior inválido: R\$ 1.000.000,00

Casos de Arredondamento:

- R\$ 10,555 $\rightarrow$ R\$ 10,56 (arredonda para cima)
- R\$ 10,554 $\rightarrow$ R\$ 10,55 (arredonda para baixo)
- R\$ 10,545 $\rightarrow$ R\$ 10,55 (HALF_UP)

2.4 Quantidade (Quantidade em Estoque)

Descrição: Representa a quantidade disponível de um produto em estoque.

Comportamentos Esperados:

- DEVE aceitar valores inteiros entre 0 e 999.999
- DEVE permitir criação de quantidade zero através de zero()
- DEVE permitir adicionar quantidade criando novo objeto
- DEVE permitir subtrair quantidade criando novo objeto
- DEVE rejeitar valores negativos com `IllegalArgumentException`
- DEVE rejeitar valores acima de 999.999 com `IllegalArgumentException`
- DEVE rejeitar subtrações que resultem em valores negativos
- DEVE rejeitar adições que excedam o valor máximo
- DEVE manter imutabilidade (operações retornam novos objetos)

Invariantes:

- Valor sempre ≥ 0
- Valor sempre ≤ 999.999
- Operações de adição e subtração não modificam o objeto original

Partições de Equivalência:

- **Válidas:**
 - 0 (limite inferior)
 - 1 a 999.998 (faixa válida)
 - 999.999 (limite superior)
- **Inválidas:**
 - Valores negativos
 - Valores acima de 999.999

Valores Limite:

- Limite inferior válido: 0
- Limite inferior inválido: -1
- Limite superior válido: 999.999
- Limite superior inválido: 1.000.000

Propriedades Matemáticas:

- Adição é comutativa: `a.adicionar(b) == b.adicionar(a)`
- Subtração é inversa de adição: `q.adicionar(n).subtrair(n) == q`
- Imutabilidade: `q.adicionar(n)` não altera `q`

2.5 Categoria (Categoria do Produto)

Descrição: Classificação do produto em categorias predefinidas.

Categorias Disponíveis:

1. ELETRONICOS - "Eletrônicos"
2. ALIMENTOS - "Alimentos"
3. VESTUARIO - "Vestuário"
4. MOVEIS - "Móveis"
5. LIVROS - "Livros"
6. BRINQUEDOS - "Brinquedos"
7. ESPORTES - "Esportes"
8. BELEZA - "Beleza"
9. AUTOMOTIVO - "Automotivo"
10. OUTROS - "Outros"

Comportamentos Esperados:

- DEVE retornar descrição legível através de `getDescricao()`
- DEVE permitir busca por descrição através de `fromDescricao(String)`
- DEVE buscar categoria ignorando diferenças de maiúsculas/minúsculas
- DEVE buscar categoria removendo espaços em branco
- DEVE rejeitar descrições nulas com `IllegalArgumentException`
- DEVE rejeitar descrições vazias com `IllegalArgumentException`
- DEVE rejeitar descrições inexistentes com `IllegalArgumentException`
- DEVE ter exatamente 10 categorias definidas

Invariantes:

- Conjunto de categorias é fixo e imutável
- Cada categoria possui uma descrição única

- Descrições nunca são nulas ou vazias

Casos de Teste:

- Busca case-insensitive: "eletrônicos", "ELETRÔNICOS", "ELETRÔNicos" → ELETRONICOS
- Busca com espaços: " Eletrônicos " → ELETRONICOS
- Busca inválida: "Categoria Inexistente" → IllegalArgumentException

2.6 Produto (Entidade Agregada)

Descrição: Entidade principal que agrega todos os value objects representando um produto completo.

Comportamentos Esperados - Criação:

- DEVE criar produto com ID gerado automaticamente através de criar()
- DEVE reconstituir produto com ID existente através de reconstituir()
- DEVE rejeitar qualquer atributo nulo com IllegalArgumentException
- DEVE manter imutabilidade de todos os atributos

Comportamentos Esperados - Atualização:

- DEVE permitir atualizar nome retornando novo produto
- DEVE permitir atualizar preço retornando novo produto
- DEVE permitir atualizar quantidade retornando novo produto
- DEVE permitir atualizar categoria retornando novo produto
- DEVE manter o mesmo ID após qualquer atualização
- DEVE manter o produto original inalterado (imutabilidade)

Comportamentos Esperados - Igualdade:

- DEVE considerar produtos iguais se possuem o mesmo ID
- DEVE considerar produtos diferentes se possuem IDs diferentes
- DEVE produzir mesmo hashCode para produtos com mesmo ID
- NÃO DEVE considerar outros atributos além do ID para igualdade

Invariantes:

- ID nunca é nulo
- Nome nunca é nulo
- Preço nunca é nulo
- Quantidade nunca é nula
- Categoria nunca é nula
- Uma vez criado, o ID não muda
- Todas as atualizações retornam novos objetos

3. Especificações de Repositório

3.1 ProdutoRepositorio (Interface)

Descrição: Contrato para operações de persistência de produtos.

Operações Especificadas:

3.1.1 salvar(Produto)

- **Pré-condição:** Produto não é nulo
- **Pós-condição:** Produto é armazenado e pode ser recuperado pelo ID
- **Exceções:** IllegalArgumentException se produto for nulo

3.1.2 buscarPorId(Produtoid)

- **Pré-condição:** ID não é nulo
- **Pós-condição:** Retorna Optional com produto se existir, vazio caso contrário
- **Exceções:** IllegalArgumentException se ID for nulo

3.1.3 buscarTodos()

- **Pré-condição:** Nenhuma
- **Pós-condição:** Retorna lista com todos os produtos (pode estar vazia)
- **Exceções:** Nenhuma

3.1.4 atualizar(Produto)

- **Pré-condição:** Produto não é nulo E produto existe no repositório
- **Pós-condição:** Produto é atualizado com novos valores
- **Exceções:**
 - IllegalArgumentException se produto for nulo
 - IllegalStateException se produto não existir

3.1.5 deletar(Produtoid)

- **Pré-condição:** ID não é nulo
- **Pós-condição:** Produto é removido se existir
- **Exceções:** IllegalArgumentException se ID for nulo

3.1.6 existe(Produtoid)

- **Pré-condição:** ID não é nulo
- **Pós-condição:** Retorna true se produto existe, false caso contrário
- **Exceções:** IllegalArgumentException se ID for nulo

3.1.7 contar()

- **Pré-condição:** Nenhuma
- **Pós-condição:** Retorna quantidade total de produtos
- **Exceções:** Nenhuma

3.2 ProdutoRepositorioMemoria (Implementação)

Descrição: Implementação em memória usando ConcurrentHashMap para thread-safety.

Comportamentos Específicos:

- DEVE usar ConcurrentHashMap para armazenamento
- DEVE garantir thread-safety em todas as operações
- DEVE retornar cópias de listas, não referências internas

- DEVE validar todos os parâmetros antes de qualquer operação
-

4. Especificações de Serviço

4.1 ProdutoService

Descrição: Camada de serviço que orquestra operações de negócio sobre produtos.

Comportamentos Esperados:

4.1.1 criar(NomeProduto, Preco, Quantidade, Categoria)

- **Pré-condições:**
 - Todos os parâmetros são válidos
 - Produto não existe no repositório
- **Pós-condições:**
 - Produto é criado com ID único
 - Produto é salvo no repositório
 - Produto pode ser recuperado pelo ID
- **Exceções:**
 - IllegalArgumentException para parâmetros inválidos
 - ProdutoDuplicadoException se produto já existe

4.1.2 buscar(Produtoid)

- **Pré-condições:** ID não é nulo
- **Pós-condições:** Retorna produto encontrado
- **Exceções:**
 - IllegalArgumentException se ID for nulo
 - ProdutoNaoEncontradoException se produto não existir

4.1.3 listarTodos()

- **Pré-condições:** Nenhuma
- **Pós-condições:** Retorna lista com todos os produtos
- **Exceções:** Nenhuma

4.1.4 atualizar(Produtoid, NomeProduto, Preco, Quantidade, Categoria)

- **Pré-condições:**
 - ID não é nulo
 - Produto existe no repositório
 - Novos valores são válidos
- **Pós-condições:**
 - Produto é atualizado com novos valores
 - ID permanece o mesmo
- **Exceções:**
 - IllegalArgumentException para parâmetros inválidos
 - ProdutoNaoEncontradoException se produto não existir

4.1.5 deletar(Produtoid)

- **Pré-condições:**
 - ID não é nulo
 - Produto existe no repositório
- **Pós-condições:**
 - Produto é removido do repositório
 - Produto não pode mais ser recuperado
- **Exceções:**
 - IllegalArgumentException se ID for nulo
 - ProdutoNaoEncontradoException se produto não existir

4.1.6 contarProdutos()

- **Pré-condições:** Nenhuma
- **Pós-condições:** Retorna quantidade total de produtos
- **Exceções:** Nenhuma

4.1.7 existe(Produtoid)

- **Pré-condições:** ID não é nulo
- **Pós-condições:** Retorna true se produto existe, false caso contrário
- **Exceções:** IllegalArgumentException se ID for nulo

5. Especificações de Interface CLI

5.1 SistemaCLI

Descrição: Interface de linha de comando para interação com o sistema.

Menu Principal - Opções:

1. Criar Produto
2. Buscar Produto
3. Listar Produtos
4. Atualizar Produto
5. Deletar Produto
6. Contar Produtos
7. Sair

Comportamentos Esperados:

5.1.1 Criar Produto

- DEVE solicitar nome, preço, quantidade e categoria
- DEVE validar entrada antes de criar
- DEVE exibir ID gerado em caso de sucesso
- DEVE exibir mensagem de erro em caso de falha
- DEVE retornar ao menu principal após operação

5.1.2 Buscar Produto

- DEVE solicitar ID do produto

- DEVE exibir todos os dados do produto se encontrado
- DEVE exibir mensagem de erro se não encontrado
- DEVE retornar ao menu principal após operação

5.1.3 Listar Produtos

- DEVE exibir lista vazia se não há produtos
- DEVE exibir todos os produtos cadastrados
- DEVE formatar dados de forma legível
- DEVE retornar ao menu principal após operação

5.1.4 Atualizar Produto

- DEVE solicitar ID do produto
- DEVE exibir dados atuais do produto
- DEVE solicitar novos valores
- DEVE validar entrada antes de atualizar
- DEVE exibir dados atualizados em caso de sucesso
- DEVE exibir mensagem de erro em caso de falha

5.1.5 Deletar Produto

- DEVE solicitar ID do produto
- DEVE confirmar deleção (implícito)
- DEVE exibir mensagem de sucesso
- DEVE exibir mensagem de erro se produto não existe

5.1.6 Contar Produtos

- DEVE exibir total de produtos cadastrados
- DEVE exibir zero se não há produtos

5.1.7 Sair

- DEVE encerrar o sistema
- DEVE fechar recursos (Scanner)

Tratamento de Erros na CLI:

- DEVE capturar todas as exceções
- DEVE exibir mensagens amigáveis ao usuário
- DEVE permitir continuar operação após erro
- NÃO DEVE expor stack traces ao usuário

6. Especificações de Exceções

6.1 ProdutoNaoEncontradoException

Descrição: Sinaliza que um produto solicitado não foi encontrado.

Comportamentos:

- DEVE estender RuntimeException

- DEVE armazenar o ProdutoId que não foi encontrado
- DEVE fornecer mensagem descritiva incluindo o ID
- DEVE permitir acesso ao ID através de getProdutoId()

6.2 ProdutoDuplicadoException

Descrição: Sinaliza tentativa de criar produto com ID já existente.

Comportamentos:

- DEVE estender RuntimeException
- DEVE armazenar o ProdutoId duplicado
- DEVE fornecer mensagem descritiva incluindo o ID
- DEVE permitir acesso ao ID através de getProdutoId()